

实验一：从零手写 Attention

Kaggle 上的 **Medical Text Dataset** 非常适合作为“从零手写 Attention”实验的数据源。该数据集包含了大量的医学文本（如病案摘要、医学论文片段），并将其分类为不同的医学领域（如 Digestive System Diseases, Cardiovascular Diseases 等）。

实验任务：基于手写 Attention 的医学文本多分类诊断

1. 实验目标

利用 [🔗 Medical Text Dataset](https://www.kaggle.com/datasets/chaitanyakck/medical-text)

（[🔗 https://www.kaggle.com/datasets/chaitanyakck/medical-text](https://www.kaggle.com/datasets/chaitanyakck/medical-text)）中的临床描述文本，通过学生自制的 **Multi-Head Attention (MHA)** 层，构建一个能够识别疾病所属领域的分类模型。也可以选择微信群发布的数据集（训练数据文件 train.json 和测试数据文件 test.json）

2. 实验核心任务

- **数据清洗与加载**：处理 train.dat 和 test.dat。将文本进行 Tokenization，并截断或填充（Padding）至固定长度（如 128 或 256）。
- **手写算子**：必须调用自己实现的 ScaledDotProductAttention 类。
- **模型构建**：搭建一个简单的 Transformer Encoder 层（包含 MHA、残差连接、LayerNorm 和 FFN），最后接一个全连接层输出疾病分类。
- **掩码应用**：实现 Padding Mask，防止模型将注意力浪费在无意义的 [PAD] 词元上。

3. 实验指南

第一步：准备数据

数据集中的文本通常包含复杂的医学词汇。

- **提示**：建议使用简单的 torchtext 或 GPT-2 Tokenizer 进行词元化。
- **标签映射**：将类别（如 1: Digestive, 2: Cardiovascular...）映射为 One-hot 或

整数索引。

第二步：实现医学语义“关联”可视化

这是本实验最具教学意义的部分。要求学生在训练完成后，随机抽取一条病历，提取其 Attention Map。

- **实验观测：**观察当输入包含“heart failure”（心力衰竭）时，Attention 头是否成功对准了“dyspnea”（呼吸困难）或“edema”（水肿）。
- **对比：**比较单头与多头注意力在医学实体关联上的差异。

第三步：模型验证与评估

- **评估指标：**使用准确率 (Accuracy) 和 F1-score 评估模型。
- **对比实验：**要求学生记录“使用缩放因子 $\frac{1}{\sqrt{d_k}}$ ”与“不使用”时，Loss 下降曲线的差异，并在报告中解释原因。

4. 实验报告要求

1. **代码实现：**提交完成代码，其中必须包含手写的 MultiHeadAttention 类代码片段。
2. **维度追踪：**绘制出数据在进入 Attention 之前、Split 头之后、以及 Concat 之后的 Tensor 维度变化图。
3. **结果分析：**展示一张医学文本的注意力热力图
4. **结交实验报告：**实验报告提交老师邮箱：sdzhao@ir.hit.edu.cn

5. 关键代码示例

```
import torch
import torch.nn as nn
import math

def scaled_dot_product_attention(q, k, v, mask=None):
    # d_k 是特征维度
    d_k = q.size(-1)
    # 1. 计算 scores: (batch, heads, seq_len, seq_len)
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)

    # 2. 应用 mask (如果是解码器阶段)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    # 3. Softmax 归一化
    p_attn = torch.softmax(scores, dim=-1)

    # 4. 加权求和: (batch, heads, seq_len, d_head)
    return torch.matmul(p_attn, v), p_attn
```